You can monitor the operation's result via the Kubernetes CLI (`kubectl get pod -l app=catalog-service`) or the Octant GUI (refer to chapter 7 for more information). Once the applications are available and ready, we can check the application logs using the CLI:

```
$ kubectl logs deployment/catalog-service
```

One of the first Spring Boot log events will tell you that the `staging` profile is enabled, just like we configured in the staging overlay via a patch.

The application is not exposed outside the cluster, but you can use the port-forwarding functionality to forward traffic from your local environment on port 9001 to the Service running in the cluster on port 80:

```
$ kubectl port-forward service/catalog-service 9001:80
```

Next, open a new Terminal window and call the application's root endpoint:

```
$ http :9001
Welcome to the book catalog from a staging Kubernetes environment!
```

The result is the customized message we defined in the application-staging.yml file for the `polar.greeting` property. That's exactly what we were expecting.

NOTE It's worth noticing that if you send a `GET` request to `:9001/books`, you'll get an empty list. In staging, we haven't enabled the `testdata` profile controlling the generation of books at startup time. We want that only in a development or test environment.

The last customization we applied to the staging overlay was the number of replicas to deploy. Let's verify that with the following command:

```
$ kubectl get pod -l app=catalog-service
```